Cover page

Declaration of Originality

Proforma

Contents

1 Introduction **6**
- 1.1 The Problem in a Question . . . 6
- 1.2 Potential Answers . . . 6
- 1.3 My Project . . . 6

2 Preparation **7**
- 2.1 Background Material . . . 7
  - 2.1.1 Tagged Memory Architectures . . . 7
  - 2.1.2 CHERI . . . 8
  - 2.1.3 A Basic Scheme . . . 9
  - 2.1.4 The Importance of a Tag Cache . . . 10
  - 2.1.5 *Efficient Tagged Memory*'s Hierarchical Scheme . . . 10
  - 2.1.6 The Morello Project Scheme . . . 12
- 2.2 My Project – A Tag Controller Simulator . . . 13
  - 2.2.1 The Workloads . . . 13
  - 2.2.2 ChampSim – An Appropriate Cache Simulator . . . 13
  - 2.2.3 Visualisation Tool . . . 14
  - 2.2.4 Requirements Analysis . . . 14
- 2.3 Starting Pont . . . 14

3 Implementation **18**
- 3.1 Implementation Overview . . . 18
- 3.2 Trace Decoder . . . 18
- 3.3 Simulator Framework . . . 18
- 3.4 Tag Cache . . . 18
- 3.5 Tag Controller Algorithms . . . 18
  - 3.5.1 Basic Scheme . . . 18
  - 3.5.2 *Efficient Tagged Memory*'s Hierarchical Scheme . . . 18
  - 3.5.3 The Morello Project Scheme . . . 18
- 3.6 Logging . . . 18
  - 3.6.1 Logging Tag Table State . . . 18
  - 3.6.2 Adding Logging to ChampSim for Tag Cache State . . . 18
- 3.7 Visualisation Tool . . . 18
- 3.8 Testing . . . 18
- 3.9 Repository Overview . . . 18

4 Evaluation **19**

5 Conclusions **20**

List of Figures

1 How the two parts of a tagged value are stored in physical memory. 7
2 Step-by-step memory access translation. 7
3 How the parts of the memory system interface. 8
4 A CHERI capability. 8
5 Where the tag cache sits in the memory system. 10
6 *Efficient Tagged Memory*'s two-level hierarchical lookup table. 11
7 The algorithm for reading tags. Memory access operations are highlighted blue. Operations internal to the tag controller are grey. 12
8 The algorithm for writing tags. 15
9 The physical layout of the hierarchical table. 16
10 The layout of the two tag caches in the Morello memory system. 16
11 A black-box diagram explaining the inputs and outputs of the pieces of the tag controller evaluation framework. 17

1 Introduction

1.1 The Problem in a Question

The paper *Efficient Tagged Memory* [1] describes how tagged memory architectures have been explored in academia and employed in industry many times with various different focuses throughout the history of computer design. Through both academic rigour and market testing they have been proven to provide system-wide security properties such as hardware capabilities [2], pointer integrity [3], watchpoints [4], and information-flow tracking [5].
The employment of a tagged memory architecture can certainly provide utility, but there are associated costs that render widespread adoption of tagged memory architectures currently infeasible. With a simplistic scheme, the number of memory accesses that workloads must perform doubles due to the CPU now having to access the tag of the value it writes, as well as the value itself, which brings an unacceptably large time overhead. Also, the table in which tags are stored comes with a constant memory consumption cost, which is particularly disadvantageous within the server-side compute industry, where every byte of DRAM is precious.

Efforts have been and are currently being made to mitigate the overheads of tagged memory. The question that this dissertation is concerned with is:

"How can we further reduce the overheads that tagged memory introduces?"

1.2 Potential Answers

Tag caches are often used in practice to hold tag data close to the CPU for faster access times. The main area in which mitigation of the memory access latency overhead is found is in increasing the 'cacheability' of the tag table in DRAM. That is, by increasing the amount of information about the tags that we store in a fast-access cache, we reduce the average latency overhead that is introduced by tagged memory's new requirement of accessing tag data, as well as value data. An effort of this kind, described in *Efficient Tagged Memory*, [cite again???] has been shown to reduce the common-case DRAM traffic overhead to "well below 1%".
Also, suppose that the tagged memory system is one which runs a hypervisor, which is a common setup for servers. Suppose further that the hypervisor has a dynamic DRAM allocator that could offer free physical address regions to its guest operating systems as and when they require it. A novel system involving a dynamically-resizing tag table that can be compressed and decompressed at runtime would allow the guest OSes to reserve and free regions of DRAM that they use for storing their tags as they execute, meaning that each OS will consume large amounts of extra space only when necessary, as opposed to all of them reserving at boot-time the maximum that they might require during any part of their execution. By increasing the 'compressibility' of the tag table, we can keep the size of the tag table in each guest OS instance closer to a minimum, given the current workloads each is performing.

1.3 My Project

These potential areas of improvement motivate the exploration and investigation of existing tag controller schemes in search of opportunities to improve upon them, to increase the 'cacheability' and 'compressibility' of the DRAM tag table. They also motivate the creation of tools that allow architects to quantitatively and qualitatively evaluate new tag controller schemes that they might design.
My project aims to explore existing tag controller implementations, as well as provide a framework for an architect to implement a controller of their own design, which provides them with qualitative and quantitative data that allows them to usefully investigate and understand its performance, for the sake of contributing to the field of research in this area, and participating in answering the question I posed earlier.

2 Preparation

This chapter explains the relevant background material required to undertake the project and to understand the dissertation. It also details the major pieces of my project, how they fit together, and what was required to produce a successful implementation.

2.1 Background Material

2.1.1 Tagged Memory Architectures

In a tagged memory architecture N-bit values in memory are notionally 'extended' with M-bit tags. Doing so can provide desirable security properties like those mentioned in the introduction. *Efficient Tagged Memory* [1] describes how a one-bit tag is sufficient to implement all of the security properties that tagged memory architectures can afford. The extension of values with tags might be implemented in one of several ways, but section III of the paper details how storing all of the tag bits in a single table at the top of DRAM is the most practical for single-bit tag (SBT) architectures; this is also the most used technique in practice.
These architectures mandate allocation of a static region of DRAM at boot-time to act as the tag table. This results in a constant physical memory consumption cost that is proportional to the size of the (taggable) address space.

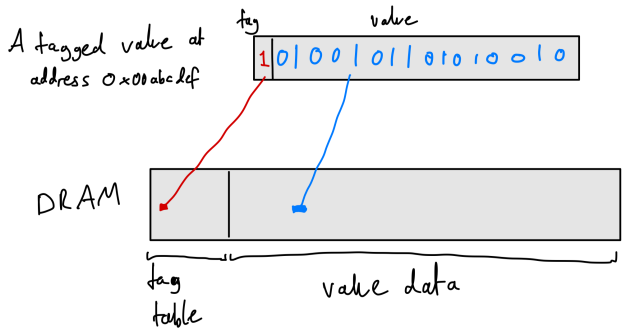

Figure 1: How the two parts of a tagged value are stored in physical memory.

To implement such a scheme in hardware, we use a tag controller. This unit is responsible for intercepting memory access requests and translating them into appropriate requests that will result in the retrieval of both the value and its tag.

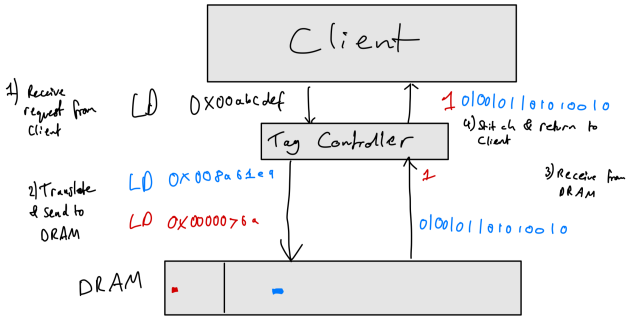

Figure 2: Step-by-step memory access translation.

My project focuses on merged-cache hierarchy architectures. In these, values in the cache are stored unified with their tag – that is, the data caches contain tags as well as values. Section IV of *Efficient Tagged Memory* explains how the alternative, split-cache hierarchy architectures, are 'unnecessary', and result in a less efficient and less practical implementation. Thus, only when handling last-level cache (LLC) misses do we need to perform such translations. The tag controller therefore interfaces with the LLC and DRAM.

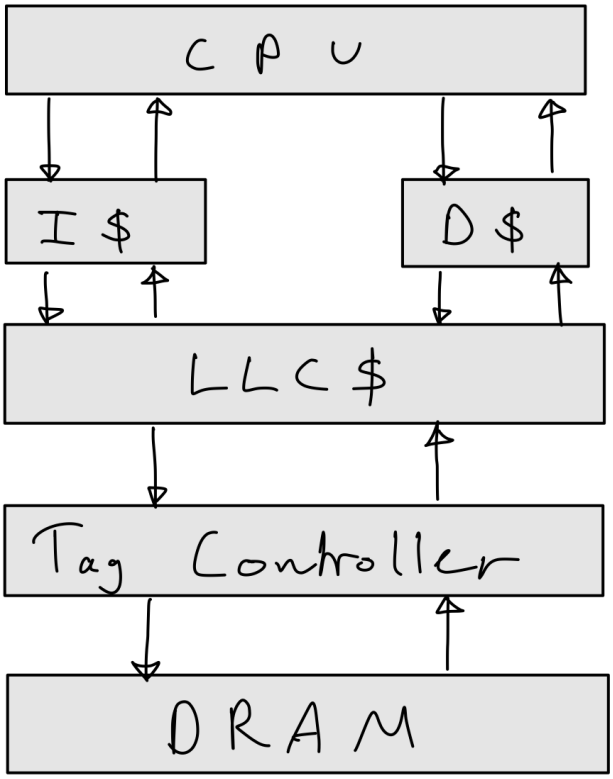

Figure 3: How the parts of the memory system interface.

2.1.2 CHERI

My project focuses on the CHERI architecture model. The model defines a single-bit tagged memory architecture, where 64-bit pointers can be replaced with 128-bit 'capabilities' that are also are given a 1-bit tag, which allows hardware-enforced security invariants to be guaranteed about these special values, and thus also about executions of programs that use them. There is more information about the CHERI model on the website for the Departement of Computer Science and Technology at the University of Cambridge [6].

Figure 4: A CHERI capability.

My project focuses particularly on the CHERI architecture as it is a speciality of the university and of my supervisor, and it is also the tagged memory architecture that I am most familiar with, as it was briefly introduced to us in IB, and I also spent some time working with it over the previous summer during my internship as a member of Arm's Morello Kernel Team. Much of my work and findings however will be transferable to other tagged memory architectures.

2.1.3 A Basic Scheme

The most basic scheme has a single flat table at the top of DRAM that contains all of the tags, and nothing more. This table contains one tag bit per 128 bits of regular memory (as a CHERI capability is 128-bits plus a 1-bit tag). The table therefore consumes the top 129^{th} of DRAM. The layout of DRAM is as depicted in figure 1.

As far as the processor and the cache systems are concerned, regular memory still begins at physical address `0x0`. The fact that the controller is mandating that the base of the *tag table* is at `0x0`, rather than the base of physical memory being here, is abstracted away by the controller's interface. For clarity, I will refer to the addresses that the processor and caches see as 'logical physical addresses', and those that the tag controller emits and DRAM sees as 'actual physical addresses'. These are both distinct from virtual addresses.
Also, note that everything here is done in terms of cachelines, as the tag controller's client is a cache – the tag controller receives logical base addresses of cachelines from the LLC, rather than those of individual values, and it must return entire cachelines to the LLC.

So, suppose the last-level cache misses the cacheline with logical base address a. It sends a request to the tag controller to retrieve the cacheline that lives at logical address a, and also its corresponding tags. The controller then uses the following formulae to calculate which actual physical addresses it needs to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{128} \tag{1}$$

$$\text{Actual Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} \tag{2}$$

Where the notation $\lfloor x \rfloor_y$ denotes the nearest multiple of y that is less than or equal to x – i.e. rounding x down to the nearest multiple of y.

For the actual base address of the value cacheline, we must take into account the offset of the value portion of DRAM. The value portion begins immediately after the end of the tag table. The tag table has size $\frac{\text{Physical Memory Size}}{128}$, thus we add this value onto the logical address of the line, a, to get the actual physical address of the line.
The actual base address of the tag cacheline will index into the tag table. In this scheme, the tags are arranged in the same order as the 128-bit values are themselves in DRAM. That is, the first bit of the tag table is the tag of the first 128-bit value in DRAM. The second bit is that of the second 128-bit value, and so on. As we assume each cacheline is the industry standard of 64 bytes, and that each tag corresponds to 128 bits (16 bytes), there are 4 tags per value cacheline. Thus, **one cacheline of tags from the tag table** holds the tags for **128 value cachelines** (see equation 3) By dividing a by 128 we get the location in the tag table that corresponds to the location of the value in the value portion of DRAM. We then must calculate the base address of the tag cacheline which this location lies in, which is done by rounding down to the nearest multiple of 64, as the tag cachelines are 64 bytes.

$$\frac{512 \text{ tags per tag cacheline}}{4 \text{ tags per value cacheline}} = 128 \text{ value cachelines covered by one tag cacheline} \tag{3}$$

2.1.4 The Importance of a Tag Cache

The tag controller then needs to access memory twice; once to retrieve the value cacheline, and once more to retrieve the tag cacheline, from which the appropriate tags are extracted and merged with the values, before returning this to its LLC client. This double access is where the hefty time overhead lies – having to access DRAM twice for every LLC miss would of course roughly double the the controller's response time. Without adding another level of data cache, we cannot avoid accessing the value cacheline in DRAM. What we can do though is introduce a new cache at this level of the memory hierarchy, just below the LLC and alongside the tag controller, that stores data from **the DRAM tag table**. This is known as the tag cache.

Now, we first check to see of the tag cacheline we want is in the tag cache. This request can be performed in parallel to the DRAM lookup of the value cacheline. If the tag cache happens to hit, then we avoid the second DRAM access entirely, and eliminate the overhead. Naturally, this motivates efforts to maximise the hit rate of the tag cache.

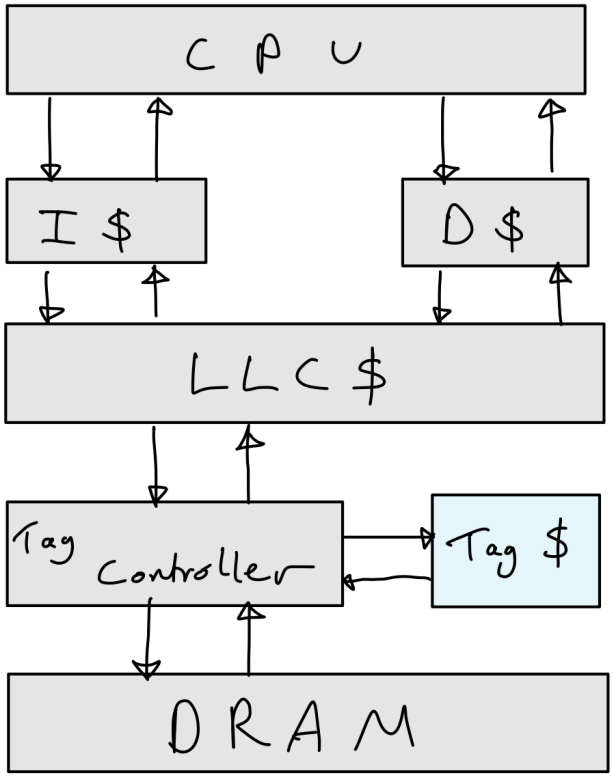

Figure 5: Where the tag cache sits in the memory system.

2.1.5 *Efficient Tagged Memory*'s Hierarchical Scheme

One of the tag controller schemes that my project focuses on is the scheme described in *Efficient Tagged Memory*. This is in part becuase of its simplicity, but also because my supervisor contributed to the paper; his familiarity with the scheme allowed him to guide me easily and effectively where necessary.

One way to increase the hit rate of the tag cache is to store more information about the tag table within it.

This can be done by simply increasing its size. However, there is an alternative – by adjusting the scheme, we can increase the 'cacheability' of the tag table. That is, we can fit more information about the tag table in the same size tag cache. Section VI of *Efficient Tagged Memory* details a tag table scheme that does just this.

This scheme uses a two-tiered hierarchical tag table. The 'leaf' table is identical to the table described in the basic scheme, with one tag bit per 128-bit value. The 'root' table contains bits that hold information about *the state of the tags in the leaf table.*
One bit in the root table corresponds to 512 bits in the leaf table – an entire cacheline of tags. The root bit is a 0 if all 512 bits in the corresponding leaf cacheline are 0, otherwise it is a 1.

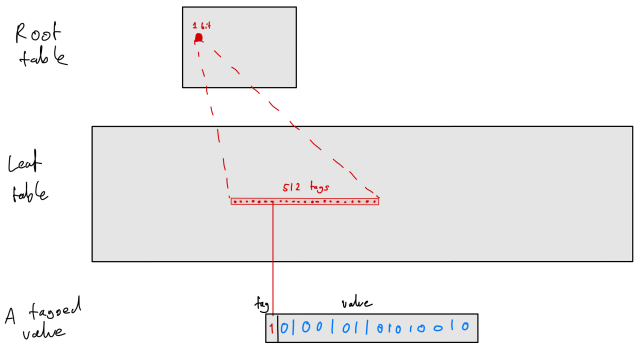

Figure 6: Efficient Tagged Memory's two-level hierarchical lookup table.

The tag controller has a new algorithm for both reading and writing the state of tags in the table. The algorithm for reading is as follows:

The algorithm for writing is as follows:

The tag cache may hold lines from either the root table or the leaf table. For each *cleared bit from the root table* that resides in the tag cache, we are essentially containing information about the state of an entire leaf cacheline whilst only consuming one bit of cache space. When executing typical CHERI workloads, it is common to find cleared root bits. It is even common to find entire cachelines of cleared root bits. This is due to the nature of these workloads – the capabilities tend to be focused into small regions of memory, rather than spread out across the entire address space.
Thus, whilst in some cases we triple the number of DRAM accesses for a given LLC miss, the paper finds that there is only a very small overall increase in DRAM traffic overhead, due to the tag cache being able to hold plenty of information about the tags, and it therefore being scarcely necessary to perform additional DRAM accesses.
Part of my project is to reproduce these results(!!! check that i actually do do this).

In DRAM, the root table lives above the leaf table. The leaf table lives above the value portion of DRAM (9).

Suppose again that the last-level cache misses the cacheline with logical base address a. The controller uses the following formulae to calculate which actual physical addresses it might need to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{65536} + \frac{\text{Physical Memory Size}}{128} \tag{4}$$

$$\text{Actual Leaf Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} + \frac{\text{Physical Memory Size}}{65536} \tag{5}$$

$$\text{Actual Root Tag Cacheline Address} = \left\lfloor \frac{a}{65536} \right\rfloor_{64} \tag{6}$$

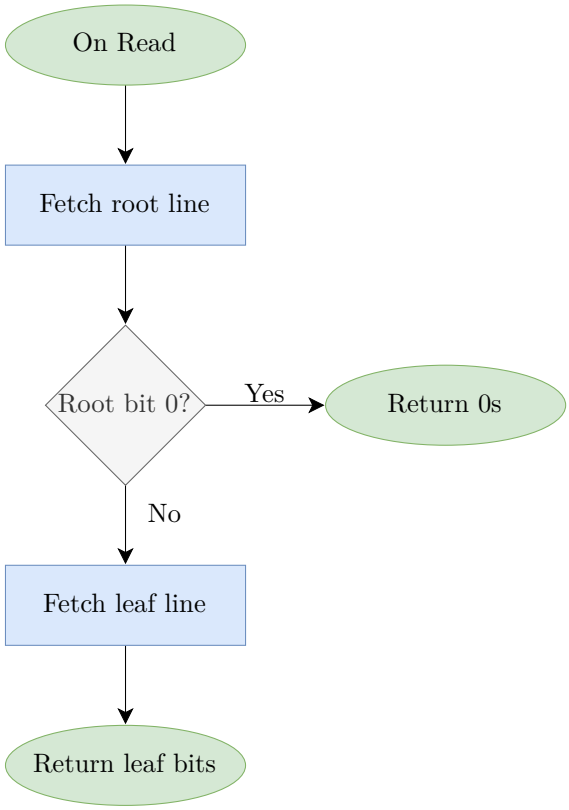

Figure 7: The algorithm for reading tags. Memory access operations are highlighted blue. Operations internal to the tag controller are grey.

The formulae for the actual base addresses of the value cacheline and the leaf tag cacheline are as they were in the simple scheme, with an additional constant of $\frac{\text{Physical Memory Size}}{65536}$ added on. This is to adjust for the offset that the root table adds. The actual base address of the root tag cacheline will index into the root table. The address is calculated by similar logic to that of the leaf tag, and equation 7 explains the presence of the constant 65536.

$$\frac{512 \text{ leaf tags per leaf cacheline}}{4 \text{ tags per value cacheline}} \times 1 \text{ leaf cacheline per root tag} \times 512 \text{ root tags per root cacheline}$$
$$= 65536 \text{ value cachelines covered by one root cacheline} \quad (7)$$

2.1.6 The Morello Project Scheme

As I mentioned prior, during my internship with Arm over the previous summer I contributed to their Morello project. Morello is a research project that defines an architecture based on the CHERI model. Another tag controller scheme that my project focuses on is the scheme used in the Morello microarchitecture. This is due to my own and my supervisor's familiarity with CHERI, and the Morello architecture and microarchitecture. There is more information about the Morello project at the project's website [7].

This scheme aims to do a similar thing to the one described in *Efficient Tagged Memory*, but in a different

way. Instead of introducing a new, more information-dense table that can be stored in the existing tag cache, it introduces a new tag cache that holds information-dense data.

This scheme uses a simple flat DRAM tag table, just like the one in the basic scheme, but it employs two tag caches. One is a cache reserved for holding tag cachelines that contain *at least one* set tag, call this the non-zero cache. The other is a cache reserved for holding tag cachelines that contain *zero* set tags – i.e. lines that are fully cleared, call this the zero cache. The former cache is just like a regular cache; for each cacheline, there is an entry in the cache that stores both the data at the address and the *address-tag** of the cacheline's base address. However, the latter cache only needs to store the address-tag of the cacheline's base address, as it must be that the entire cacheline contains zeroes, given that it is in the zero cache. Requests can be issued to and handled by the two caches in parallel, further mitigating overhead.

* The *tag of a cacheline's base address* is the upper K bits of the base address of the cacheline (for some K defined by various system parameters). This is the academic-standard name for this value, but it unfortunately clashes with the tagged memory *tags*. To disambiguate, I use the term *address-tag* hereon to refer to the former, and simply *tag* for the latter.

As the tag table has not changed from the simple scheme, the formulae used by the controller for translating addresses are the same (equations 1 and 2).

2.2 My Project – A Tag Controller Simulator

The objective of my project was to develop a tag controller simulator in C++. When given a workload it should simulate the memory accesses that a specified tag controller algorithm would result in. The idea is that this simulator can be leveraged to provide insight into the performance and inner workings of the tag controller algorithm being simulated, providing useful information to an architect.

2.2.1 The Workloads

A former student had previously created a simulator that consumes a program binary and simulates the execution of the binary on a CHERI architecture. The simulator outputs trace files that log the last-level cache misses that occurred during the simulated execution.

I have been given access to a collection of trace files by my supervisor. These files were logged whilst running various benchmark binaries. The simulator I developed takes these trace files as input. As the tag controller only receives information about LLC misses, the trace files contain sufficient information to determine which accesses would be made by the simulated tag controller algorithm if it were to run the given workload.

2.2.2 ChampSim – An Appropriate Cache Simulator

what is champsim and why is it a good choice to integrate into the proj

ChampSim is an open-source trace based cache simulator written in C++. There is more information about ChampSim at the ChampSim GitHub profile [8].

My simulator interfaces with ChampSim, having it handle the simulation of the tag cache(s). My simulator provides ChampSim with a trace file of memory accesses that the simulated tag controller makes during its execution of the workload. ChampSim then provides the user with metrics regarding cache access frequency, and hit ratio. The user can specify parameters of the tag cache such as cache size, number of ways.
A reason why ChampSim is a good choice of cache simulator is that it has built-in cache prefetchers which can be enabled or disabled simply by adjusting a configuration file. This lets an architect see how much difference adding a prefetcher to their tag cache would make to performance.

I opted to use an existing cache simulator instead of developing my own because the focus of this project is on investigating tag controllers, as opposed to caches in general, and my supervisor informed me that there are likely many opportunities for bugs to arise when developing a cache simulator of my own. Thus it is a more appropriate use of my time to focus more on tag controller research rather than debugging software that has already been written.

2.2.3 Visualisation Tool

It would be useful for an architect to be able to somehow visualise how the state of the tag table and the tag cache changes as the workload executes. Having this insight would allow !!!.

As part of my project, I developed a tool that allows a user to essentially take 'snapshots' of the tag cache at chosen points of workload execution, and view them afterward. The images depict the state of the tag table and the tag cache at that point of execution.

2.2.4 Requirements Analysis

There are several requirements that a successful simulator must meet:

- It must be able to decode the LLC miss input trace file format.
- It must be able to correctly simulate the sequence of memory accesses that the simulated tag controller algorithm would perform when executing the given workload.
- It must be able to produce an accurate output trace file of these accesses that is decodable by ChampSim.

On top of this, for the visualisation tool to be functional and useful, the following are required:

- My simulator is able to output the state of all tags at given points of execution.
- I can extract the state of the tag cache from ChampSim at given points of execution.
- The visualisation tool can read in both types of captured state and produce an image that accurately depicts it all.

2.3 Starting Pont

The following bullet points detail relevant work completed and preparations made before beginning the project:

- I had read the paper *Efficient Tagged Memory*.
- As I have mentioned, over the summer of 2023 I interned at Arm in the Morello Kernel Team, a team working to port the Linux kernel to Arm's Morello architecture, which is based on the CHERI model. This gave me a chance to get more familiar with the CHERI model, and provided me with an understanding of how a CHERI tagged memory architecture might structure its pointers, and how tag shadowspace might be laid out and accessed in DRAM.
- My project uses ChampSim, the open-source cache simulator. I also extend this program by adding logging support, by branching from commit #2b8d3fc28abb6072d7675228418fa7cfe862d4dc. I took no part in contributing to any versions of ChampSim prior to the commit.

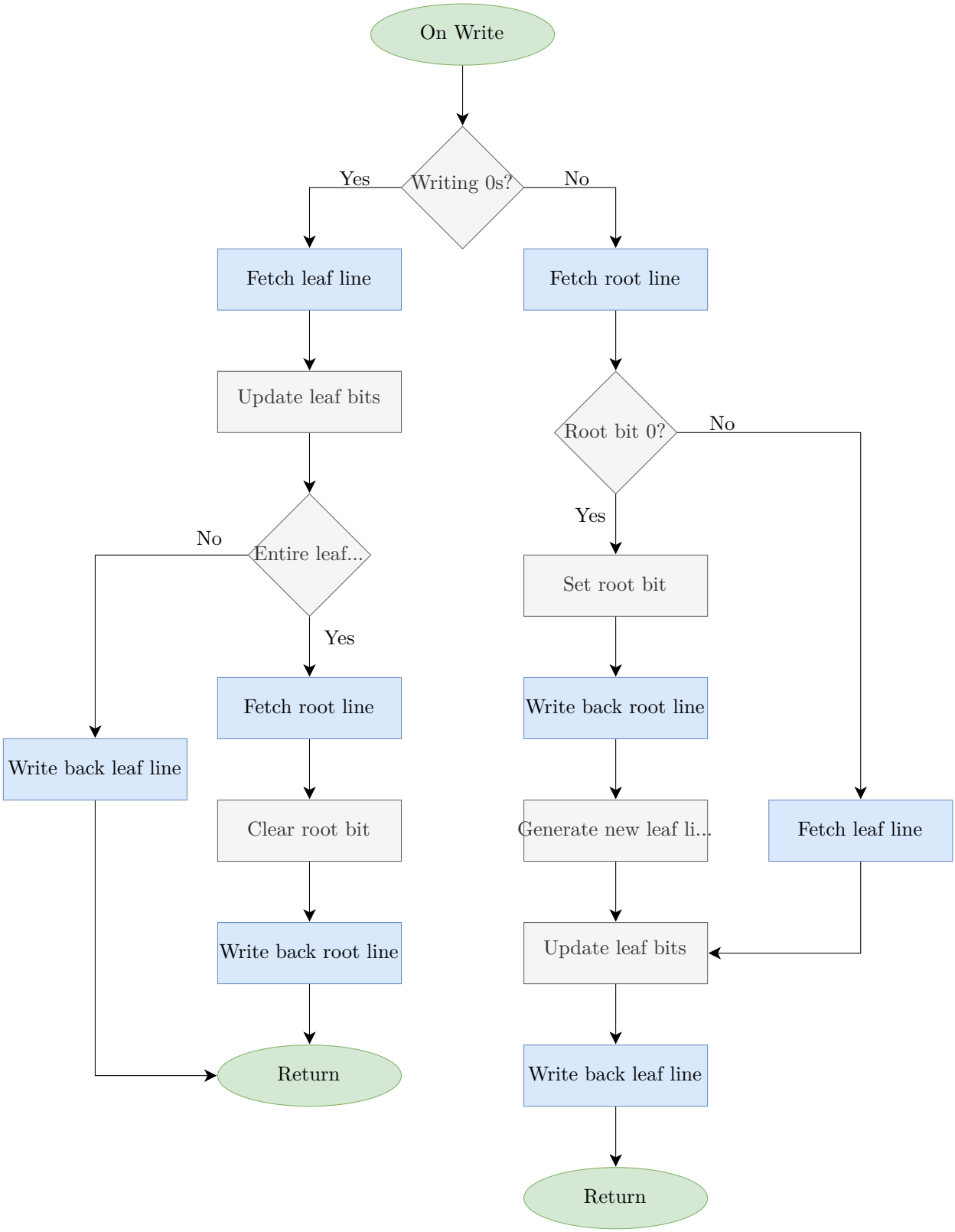

Figure 8: The algorithm for writing tags.

Figure 9: The physical layout of the hierarchical table.

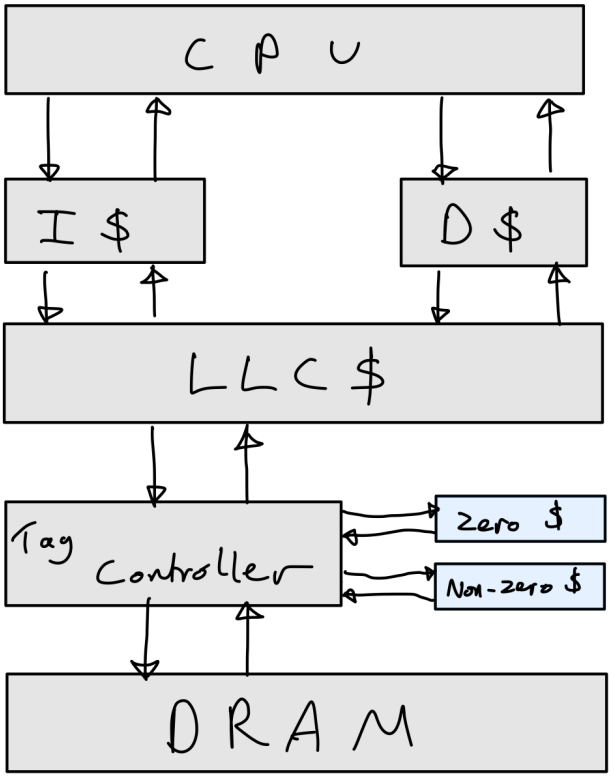

Figure 10: The layout of the two tag caches in the Morello memory system.

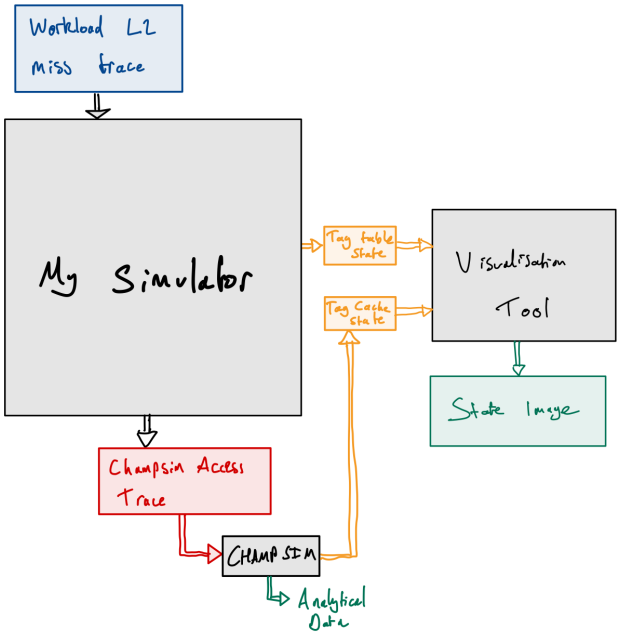

Figure 11: A black-box diagram explaining the inputs and outputs of the pieces of the tag controller evaluation framework.

3 Implementation

This chapter !!!.

3.1 Implementation Overview

3.2 Trace Decoder

3.3 Simulator Framework

3.4 Tag Cache

3.5 Tag Controller Algorithms

3.5.1 Basic Scheme

3.5.2 *Efficient Tagged Memory*'s Hierarchical Scheme

3.5.3 The Morello Project Scheme

3.6 Logging

3.6.1 Logging Tag Table State

3.6.2 Adding Logging to ChampSim for Tag Cache State

3.7 Visualisation Tool

3.8 Testing

* kept the cache a separate logical entity so it could be tested in isloation * used CMake project so I could easily add a test framework into the project

3.9 Repository Overview

4 Evaluation

5 Conclusions

References

[1] Alexandre Joannou et al. “Efficient tagged memory”. In: *2017 IEEE International Conference on Computer Design (ICCD)*. IEEE. 2017, pp. 641–648.

[2] Brian E. Clark and Michael J. Corrigan. “Application System/400 performance characteristics”. In: *IBM Systems Journal* 28.3 (1989), pp. 407–423.

[3] Jedidiah R Crandall and Frederic T Chong. “Minos: Control data attack prevention orthogonal to memory model”. In: *37th International Symposium on Microarchitecture (MICRO-37'04)*. IEEE. 2004, pp. 221–232.

[4] Joseph L Greathouse et al. “A case for unlimited watchpoints”. In: *ACM SIGPLAN Notices* 47.4 (2012), pp. 159–172.

[5] G Edward Suh et al. “Secure program execution via dynamic information flow tracking”. In: *ACM Sigplan Notices* 39.11 (2004), pp. 85–96.

[6] *https://www.cl.cam.ac.uk/research/security/ctsrd/cheri/*.

[7] *https://www.morello-project.org/*.

[8] *https://github.com/ChampSim/*.

Appendices

Index (Optional)

Project Proposal